
The Case For WAL-Logging `pg_upgrade`

Bohyun Lee (patch [GitHub repo](#))

Problem of stock `pg_upgrade`. Current `pg_upgrade` writes the files of the new cluster directly, emitting no WAL for the changes it makes. This creates two fundamental limitations. The first is a **replication gap**. Since the upgrade never enters the WAL stream, standbys cannot adopt it through replication and must be taken offline and resynchronized from the primary via `rsync` [1]. The second is a **durability gap** in the backup chain. The upgraded cluster begins with a new WAL history that is disconnected from the history of the old cluster. Consequently, if the upgraded primary fails before a new base backup is taken, transactions committed after the upgrade cannot be recovered. This window may remain open for a long time, if the database is large and taking a backup requires a long time.

Closing replication and durability gaps. The patch closes both gaps by atomically WAL-logging the after-images generated by `pg_upgrade` upon successful completion. The upgrade becomes part of the WAL stream and can be propagated to standbys through standard streaming replication, eliminating the need for offline `rsync`-based resynchronization. Furthermore, because the upgraded cluster continues the existing WAL history rather than creating a new one, the upgrade and all subsequent modifications remain recoverable through WAL replay, preserving durability even before a new base backup is taken.

Workflow with primary/standby with the proposed interface. (1) `pg_upgrade --wal-upgrade-signal-handoff` triggers the primary and standbys to shut down. (2) `pg_upgrade --wal-upgrade` performs the upgrade (3) *captures it as WAL* and shuts down. (4) The primary goes live on restart. (5) Each standby streams and replays the upgrade over standard replication, (6) becoming a hot standby once WALs are fully replayed.

Recovery workflow with primary/backup with the proposed interface. (1) Restore the last pre-upgrade base backup. (2) PITR-replay archived WAL up to the upgrade point. (3) Restart on the new version and replay the upgrade WAL. (4) Keep replaying the new-version WAL to recover the post-upgrade transactions. (5) Promote, fully recovered without new base backup.

Expected benefits. Upgrading a replicated cluster reduces to upgrading the primary with minimal cluster coordination. Standbys catch up by themselves over their existing replication link, without offline `rsync` or a rebuild. It also guarantees durability of post-upgrade modifications even without a new base backup. Such benefits strongly suggest that perhaps `--wal-upgrade` should even be the *default behavior*.

1. Proposal Overview

2. [Proposed Interface](#)
3. [Primary-standby Workflow with the New Interface](#)
4. [How --wal-upgrade Enables Continuous Archiving](#)
5. [Added TAP Tests](#)
6. [Example Use Case: k8s Zalando \(Spilo\)](#)

2. Proposed Interface

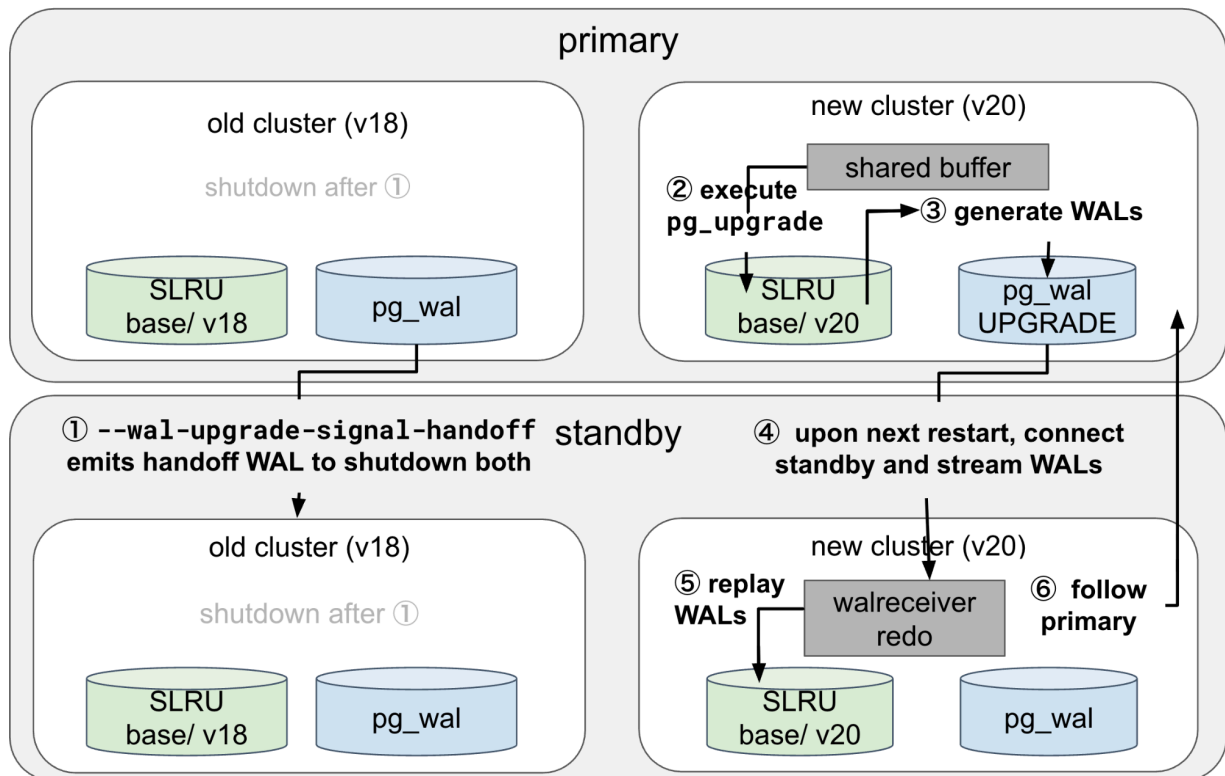
Interface. The feature is exposed through two `pg_upgrade` subcommands. Both share the `--wal-upgrade` prefix to clearly distinguish them from the stock `pg_upgrade` workflow. In a primary-standby deployment, both commands are required to upgrade both primary and standby. In deployments that rely on archived backups, only `--wal-upgrade` is required.

`pg_upgrade --wal-upgrade -d <old> -D <new>` This command is invoked like the stock `pg_upgrade`, using the same old and new data directory arguments and supporting the same transfer modes. It produces a fully upgraded cluster on disk and leaves it shut down. On the next restart, the upgraded cluster starts normally in read-write mode. Unlike the stock workflow, the upgrade is also captured in WAL once upgrade execution completes. The old cluster remains intact, while the upgrade WALs and post-upgrade WALs are streamed to standbys through standard replication.

`pg_upgrade --wal-upgrade-signal-handoff -b <old-bin> -d <old>` This command prepares existing standbys and is executed once on the running primary before the upgrade. The primary emits a handoff WAL record and then shuts down. The handoff record instructs standbys, over their existing replication connections, to stop cleanly at the upgrade point so they can transition to the upgraded cluster. During this step, client backends are terminated and the primary is fast-stopped, so no user WAL follows the handoff record.

What if the user wants to roll back? A user may wish to evaluate the upgraded version before fully committing to it, a workflow that stock Postgres also implicitly supports. This is also supported by the proposed interface. There are no behavioral changes in the primary-backup scenario. With standbys, the user can simply defer `--wal-upgrade-signal-handoff` and run only `--wal-upgrade` on the primary, leaving the standbys unchanged. If the upgrade is accepted, `--wal-upgrade-signal-handoff` can be issued afterward.

3. Primary-standby Workflow with the New Interface



① `--wal-upgrade-signal-handoff` writes a handoff record into the live v18 primary's WAL to signal standby and shuts down the primary. Standbys replay it and shut down cleanly at the boundary.

② `pg_upgrade --wal-upgrade` runs an ordinary in-place upgrade, writing the v20 base/ and SLRU files.

③ The after-images of all files `pg_upgrade` modified are then captured as FPI WAL records, emitting the entire upgrade as a WAL window (CN..COMPLETE) into the v20 `pg_wal`, where a retention slot pins it. `pg_upgrade` then shuts the cluster down cleanly. On the operator's next `pg_ctl start`, the primary skips replay and comes up read-write. The v18 cluster stays untouched.

④ A fresh v20 skeleton auto-fetches the window anchor and streams the CN..COMPLETE window from the primary.

⑤ Standby's startup/redo process replays the `UPGRADE_*` records, reconstructing the v20 base/ from WAL alone.

⑥ At COMPLETE it keeps streaming and follows the primary as a hot standby. No rsync, no base backup.

4. How `--wal-upgrade` Enables Continuous Archiving

Assume a deployment consisting of a single primary server, daily base backups, and continuous WAL archiving to separate storage. This scenario is used to illustrate how the stock `pg_upgrade` introduces a durability gap and how the proposed patch eliminates it.

The gap caused during the upgrade process. The upgrade initializes the new cluster with a fresh system identifier and a new WAL history. This breaks WAL/LSN continuity at the upgrade boundary, preventing a base backup taken before the upgrade from being replayed into the upgraded cluster; it can restore only up to the old cluster before upgrade. Until a new base backup of the upgraded cluster is created and archived, no valid backup exists for recovery of the committed transactions processed in the new cluster after upgrade.

How `--wal-upgrade` addresses this.

- ① **Capture.** `pg_upgrade --wal-upgrade` runs the normal upgrade, then captures the after-images of every modification it produced as full-page-image WAL records. These are emitted as replayable WALs on the new cluster's log stream. The new cluster keeps a fresh system identifier, `pg_control` and the window's WAL simply agree on it.
- ② **Archive.** All the generated upgrade WALs are archived like any other WAL. The archive therefore holds one continuous replay source spanning the upgrade.
- ③ **Recovery phase 1 (old binary).** On loss of the upgraded cluster, the recovery process can restore the last pre-upgrade base backup and, using `restore_command`, replay archived WAL forward to the upgrade boundary, stopping there without promoting. This recovers every pre-upgrade transaction, on the old version, into the restored data directory.
- ④ **Recovery phase 2 (new binary).** The new-version cluster is then started on that same data directory. Once it detects the recovery signal and the staged upgrade WALs, it clears the version-compatibility gate by writing a fresh new-version control file, re-anchors recovery at the CN checkpoint, and replays the `UPGRADE_*` window to reconstruct the upgraded cluster. It then naturally continues applying the archived post-upgrade WAL, recovering the transactions committed after go-live, and promotes.

5. Added TAP Tests

To verify these scenarios, the patch adds two TAP tests. `010_wal_upgrade.pl` confirms that the patch resolves the durability gap for primary-backup setups, while `011_wal_upgrade_standby.pl` validates whether the upgrade can be propagated by replaying WALs on a standby.

Added TAP test 1: `010_wal_upgrade.pl`

Phase 1: Setup and upgrade.

1. Init an old cluster with `archive_mode=on` and an atomic `archive_command`. Load pre-upgrade data, and take a base backup (`pitr_base`).
2. Run `pg_upgrade --wal-upgrade --initdb` to generate the upgrade WALs. The new cluster inherits `archive_command` and auto-serves.
3. Record the old cluster's final LSN (`old_end_lsn` = the upgrade boundary). From `pg_waldump` of the archive, the segments holding upgrade WALs.
4. Execute post-upgrade transactions, and poll until they are archived.

Phase 2: forcing failure.

5. Stop('immediate') and `rmtree` the new cluster's data directory, only the pre-upgrade base backup and the archive survive.

Phase 3: recovery of the old cluster.

6. `Init_from_backup` the pre-upgrade base backup into a fresh node.
7. Configure `restore_command`, `recovery_target_lsn` = `old_end_lsn`, `recovery_target_action` = `shutdown`, and touch the recovery signal
8. Drive with `raw pg_ctl`. This replays to the boundary and shuts down without promoting.
9. Assert the "shutdown at recovery target" log.

Phase 4: recovery of the new cluster.

10. Stage the upgrade WAL containing segments from the archive into `pg_wal`.
11. Rewrite the config: strip the `recovery_target_*` settings, keep the same `restore_command`, and set `recovery_target_timeline` = `latest`.
12. Touch `recovery.signal` and `pg_upgrade_recovery.signal`.
13. Start the new binary.
14. Assert: after `pg_is_in_recovery()` returns false, the recovered cluster holds both the pre- and post-upgrade rows (800), and `only_on_new` (=42) exists, then the post-upgrade transactions survived with no new base backup.

Added TAP test 2. 011_wal_upgrade_standby.pl

Phase 1: Old primary with an old standby.

1. Start an old primary configured for replication and load data.
2. Bring up a real old-version standby from a base backup of it, streaming over replication.
3. Wait for catch-up and assert it is a hot standby (`pg_is_in_recovery = t`) holding the pre-upgrade data (2000 rows).

Phase 2: Signal handoff.

4. Run `pg_upgrade --wal-upgrade-signal-handoff` against the live old primary; it writes an `XLOG_UPGRADE_HANDOFF` record into the primary's WAL and fast-stops the primary at that point.
5. Assert the old standby replays the handoff and shuts down cleanly at the boundary without promoting (the "reached pg_upgrade handoff on standby" log message, and its postmaster gone).

Phase 3: Upgrade.

6. Run `pg_upgrade --wal-upgrade --initdb` to produce the new primary, which auto-serves.
7. Assert the data is preserved and that the physical retention slot pinning the upgrade window is present, so the upgrade WAL is streamable.

Phase 4: New standby streams the window.

8. The operator stands up a fresh new-cluster standby against the upgraded primary by setting `primary_conninfo`, `standby.signal`, and the `pg_upgrade_stream.signal` sentinel. An ordinary standby (with `primary_conninfo` but no sentinel) is left untouched and streams normally.
9. On start, the standby does the rest on its own: it auto-fetches the window anchor over its replication connection to the primary, and streams and replays the upgrade WALs.
10. The test waits for convergence and asserts it came up as a hot standby (`pg_is_in_recovery = t`) whose data matches the upgraded primary.

Phase 5: Negative testing.

11. Start a plain new cluster with no `primary_conninfo` and no local window.
12. Assert it comes up as an ordinary, empty live cluster (not in recovery, no upgrade applied) to confirm there is no silent alternative delivery path (e.g. via the archive).

6. Example Use Case: k8s Zalando Spilo

As an example use case, the Zalando Spilo environment is considered. Spilo is the PostgreSQL appliance underlying the Zalando Patroni-based PostgreSQL operator and performs major-version upgrades through its [inplace_upgrade.py](#) procedure. In its unmodified form this procedure upgrades the primary and then reconstructs each streaming replica by transferring the entire upgraded data directory via `rsync`, requiring a complete physical copy for every replica on each upgrade.

Spilo integration workflow. The upgrade procedure is modified to adopt our interface, guarded by an opt-in `USE_WAL_UPGRADE` flag (source available [here](#); see comments labeled 'LEE'). Although this modification has not been evaluated on a live Kubernetes deployment, the objective is to illustrate the operator workflow around `pg_upgrade` rather than Kubernetes-specific behavior. The replica-handling workflow is changed as follows:

1. **Handoff.** Before the primary is shut down, `--wal-upgrade-signal-handoff` is issued against the still-live primary, so the streaming replicas stop deterministically at a well-defined point in the WAL; the same signal shuts the primary down at that point.
2. **Capture.** The upgrade then runs with `--wal-upgrade`, recording the transformation as a replayable WAL window on the primary.
3. **Removal of physical re-imaging.** The original `rsync`-based copy path (`rsyncd` startup, per-replica CHECKPOINT, and directory transfer) is bypassed entirely.
4. **Replay.** Replicas reach the new major version by replaying the upgrade window over the existing replication channel, exactly as with ordinary streaming replication.